```
import torch
import torchvision

print("PyTorch version:", torch.__version__)
print("Torchvision version:", torchvision.__version__)
print("GPU available:", torch.cuda.is_available())

if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
```

```
PyTorch version: 2.11.0+cu128
Torchvision version: 0.26.0+cu128
GPU available: True
GPU: Tesla T4
```

```
import os
import shutil
import random
import numpy as np
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
import torch.optim as optim

from torch.utils.data import DataLoader
from torchvision import datasets, transforms, models

print("All libraries imported successfully!")
```

```
All libraries imported successfully!
```

```
from torchvision.datasets import Caltech101

dataset_root = "/content/caltech101"

caltech_dataset = Caltech101(
    root=dataset_root,
    download=True
)

print("Dataset downloaded successfully!")
print("Number of images:", len(caltech_dataset))
print("Number of classes:", len(caltech_dataset.categories))
```

```
100%|██████████| 137M/137M [00:01<00:00, 110MB/s]
Dataset downloaded successfully!
Number of images: 8677
Number of classes: 101
```

```
print("First 20 classes:")
```

```
for i, class_name in enumerate(caltech_dataset.categories[:20]):
    print(i, class_name)
```

```
First 20 classes:
0 Faces
1 Faces_easy
2 Leopards
3 Motorbikes
4 accordion
5 airplanes
6 anchor
7 ant
8 barrel
9 bass
10 beaver
11 binocular
12 bonsai
13 brain
14 brontosaurus
15 buddha
16 butterfly
17 camera
18 cannon
19 car_side
```

```
import os
import shutil
import random

source_root = os.path.join(dataset_root, "caltech101", "101_ObjectCategories

base_dir = "/content/caltech_split"

train_dir = os.path.join(base_dir, "train")
valid_dir = os.path.join(base_dir, "valid")
test_dir = os.path.join(base_dir, "test")

# Remove old split if it exists
if os.path.exists(base_dir):
    shutil.rmtree(base_dir)

os.makedirs(train_dir)
os.makedirs(valid_dir)
os.makedirs(test_dir)

# Random seed for reproducibility
random.seed(42)

# Get all classes
classes = sorted([
    d for d in os.listdir(source_root)
    if os.path.isdir(os.path.join(source_root, d))
])

print("Number of classes:", len(classes))
```

```
# Create folders and split images
for class_name in classes:

    class_path = os.path.join(source_root, class_name)

    images = [
        f for f in os.listdir(class_path)
        if f.lower().endswith((".jpg", ".jpeg", ".png"))
    ]

    random.shuffle(images)

    n = len(images)

    train_end = int(0.50 * n)
    valid_end = int(0.75 * n)

    train_images = images[:train_end]
    valid_images = images[train_end:valid_end]
    test_images = images[valid_end:]

    for folder in [train_dir, valid_dir, test_dir]:
        os.makedirs(os.path.join(folder, class_name), exist_ok=True)

    for image in train_images:
        shutil.copy2(
            os.path.join(class_path, image),
            os.path.join(train_dir, class_name, image)
        )

    for image in valid_images:
        shutil.copy2(
            os.path.join(class_path, image),
            os.path.join(valid_dir, class_name, image)
        )

    for image in test_images:
        shutil.copy2(
            os.path.join(class_path, image),
            os.path.join(test_dir, class_name, image)
        )

print("Dataset split completed!")
print("Train:", train_dir)
print("Validation:", valid_dir)
print("Test:", test_dir)
```

```
Number of classes: 102
Dataset split completed!
Train: /content/caltech_split/train
Validation: /content/caltech_split/valid
Test: /content/caltech_split/test
```

```
def count_images(directory):
    count = 0
```

```
    for root, dirs, files in os.walk(directory):
        for file in files:
            if file.lower().endswith((".jpg", ".jpeg", ".png")):
                count += 1

    return count


print("Training images:", count_images(train_dir))
print("Validation images:", count_images(valid_dir))
print("Testing images:", count_images(test_dir))
```

```
Training images: 4543
Validation images: 2278
Testing images: 2323
```

```
image_transforms = {

    "train": transforms.Compose([
        transforms.RandomResizedCrop(
            size=256,
            scale=(0.8, 1.0)
        ),

        transforms.RandomRotation(
            degrees=15
        ),

        transforms.ColorJitter(),

        transforms.RandomHorizontalFlip(),

        transforms.CenterCrop(
            size=224
        ),

        transforms.ToTensor(),

        transforms.Normalize(
            [0.485, 0.456, 0.406],
            [0.229, 0.224, 0.225]
        )
    ]),

    "valid": transforms.Compose([
        transforms.Resize(
            size=256
        ),

        transforms.CenterCrop(
            size=224
        ),
```

```
        transforms.ToTensor(),

        transforms.Normalize(
            [0.485, 0.456, 0.406],
            [0.229, 0.224, 0.225]
        )
    ]),

    "test": transforms.Compose([
        transforms.Resize(
            size=256
        ),

        transforms.CenterCrop(
            size=224
        ),

        transforms.ToTensor(),

        transforms.Normalize(
            [0.485, 0.456, 0.406],
            [0.229, 0.224, 0.225]
        )
    ])
}

print("Transforms created successfully!")
```

```
Transforms created successfully!
```

```
data = {

    "train": datasets.ImageFolder(
        root=train_dir,
        transform=image_transforms["train"]
    ),

    "valid": datasets.ImageFolder(
        root=valid_dir,
        transform=image_transforms["valid"]
    ),

    "test": datasets.ImageFolder(
        root=test_dir,
        transform=image_transforms["test"]
    )
}

print("Datasets created successfully!")

print("Training:", len(data["train"]))
print("Validation:", len(data["valid"]))
print("Testing:", len(data["test"]))
```

```
Datasets created successfully!
Training: 4543
Validation: 2278
Testing: 2323
```

```
batch_size = 32

dataloaders = {

    "train": DataLoader(
        data["train"],
        batch_size=batch_size,
        shuffle=True,
        num_workers=2,
        pin_memory=True
    ),

    "valid": DataLoader(
        data["valid"],
        batch_size=batch_size,
        shuffle=False,
        num_workers=2,
        pin_memory=True
    ),

    "test": DataLoader(
        data["test"],
        batch_size=batch_size,
        shuffle=False,
        num_workers=2,
        pin_memory=True
    )
}

print("DataLoaders created successfully!")
```

```
DataLoaders created successfully!
```

```
class_names = data["train"].classes
n_classes = len(class_names)

print("Number of classes:", n_classes)
print("First 20 classes:")

for i, name in enumerate(class_names[:20]):
    print(i, name)
```

```
Number of classes: 102
First 20 classes:
0 BACKGROUND_Google
1 Faces
2 Faces_easy
3 Leopards
4 Motorbikes
```

```
5 accordion
6 airplanes
7 anchor
8 ant
9 barrel
10 bass
11 beaver
12 binocular
13 bonsai
14 brain
15 brontosaurus
16 buddha
17 butterfly
18 camera
19 cannon
```

```
images, labels = next(iter(dataloaders["train"]))

plt.figure(figsize=(12, 8))

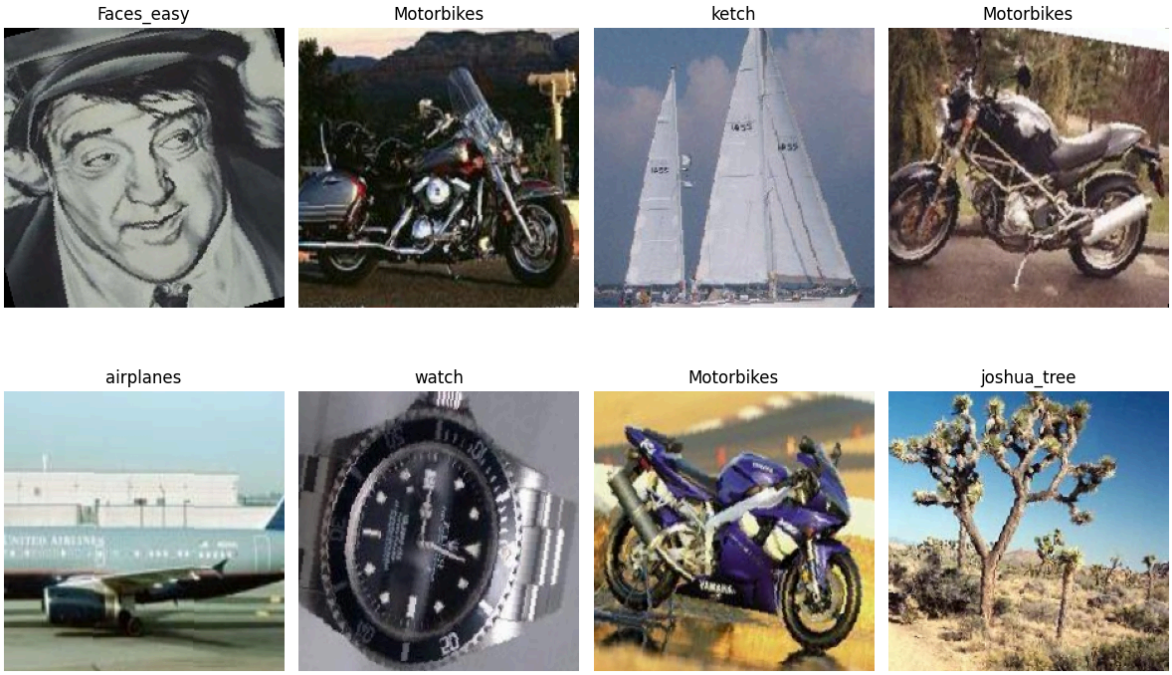
for i in range(8):
    image = images[i].permute(1, 2, 0).numpy()

    # Undo normalization approximately
    mean = np.array([0.485, 0.456, 0.406])
    std = np.array([0.229, 0.224, 0.225])

    image = image * std + mean
    image = np.clip(image, 0, 1)

    plt.subplot(2, 4, i + 1)
    plt.imshow(image)
    plt.title(class_names[labels[i]])
    plt.axis("off")

plt.tight_layout()
plt.show()
```




```
device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)

print("Using device:", device)
```

```
Using device: cuda
```

```
weights = models.VGG16_Weights.DEFAULT

model = models.vgg16(weights=weights)

print("Pre-trained VGG-16 loaded successfully!")
```

```
Downloading: "https://download.pytorch.org/models/vgg16-397923af.pth" to /roo
100%|██████████| 528M/528M [00:04<00:00, 125MB/s]
```

```
Pre-trained VGG-16 loaded successfully!
```

```
for param in model.parameters():
    param.requires_grad = False

print("All VGG-16 parameters have been frozen.")
```

```
All VGG-16 parameters have been frozen.
```

```
n_inputs = model.classifier[6].in_features

model.classifier[6] = nn.Sequential(
    nn.Linear(n_inputs, 256),
    nn.ReLU(),
    nn.Dropout(0.4),
    nn.Linear(256, n_classes),
    nn.LogSoftmax(dim=1)
)

print("Custom classifier added successfully!")
print(model.classifier)
```

```
Custom classifier added successfully!
Sequential(
  (0): Linear(in_features=25088, out_features=4096, bias=True)
  (1): ReLU(inplace=True)
  (2): Dropout(p=0.5, inplace=False)
  (3): Linear(in_features=4096, out_features=4096, bias=True)
  (4): ReLU(inplace=True)
  (5): Dropout(p=0.5, inplace=False)
  (6): Sequential(
    (0): Linear(in_features=4096, out_features=256, bias=True)
    (1): ReLU()
    (2): Dropout(p=0.4, inplace=False)
    (3): Linear(in_features=256, out_features=102, bias=True)
    (4): LogSoftmax(dim=1)
  )
)
```

```
model = model.to(device)

print("Model moved to:", device)
```

```
Model moved to: cuda
```

```
print("Trainable parameters:")

for name, param in model.named_parameters():

    if param.requires_grad:
        print(name)
```

```
Trainable parameters:
classifier.6.0.weight
```

```
classifier.6.0.bias
classifier.6.3.weight
classifier.6.3.bias
```

```
criterion = nn.NLLLoss()

optimizer = optim.Adam(
    filter(lambda p: p.requires_grad, model.parameters()),
    lr=0.001
)

print("Loss function and optimizer initialized.")
```

```
Loss function and optimizer initialized.
```

```
num_epochs = 3

train_losses = []
valid_losses = []

train_accuracies = []
valid_accuracies = []

best_valid_loss = float("inf")
patience = 2
patience_counter = 0

for epoch in range(num_epochs):

    # -------------------------
    # Training
    # -------------------------
    model.train()

    running_loss = 0.0
    correct = 0
    total = 0

    for images, targets in dataloaders["train"]:

        images = images.to(device)
        targets = targets.to(device)

        optimizer.zero_grad()

        outputs = model(images)

        loss = criterion(outputs, targets)

        loss.backward()

        optimizer.step()

        running_loss += loss.item() * images.size(0)
```

```
        _, predicted = torch.max(outputs, 1)

        total += targets.size(0)
        correct += (predicted == targets).sum().item()

    epoch_train_loss = running_loss / total
    epoch_train_accuracy = correct / total

    # -------------------------
    # Validation
    # -------------------------
    model.eval()

    running_loss = 0.0
    correct = 0
    total = 0

    with torch.no_grad():

        for images, targets in dataloaders["valid"]:

            images = images.to(device)
            targets = targets.to(device)

            outputs = model(images)

            loss = criterion(outputs, targets)

            running_loss += loss.item() * images.size(0)

            _, predicted = torch.max(outputs, 1)

            total += targets.size(0)
            correct += (predicted == targets).sum().item()

    epoch_valid_loss = running_loss / total
    epoch_valid_accuracy = correct / total

    train_losses.append(epoch_train_loss)
    valid_losses.append(epoch_valid_loss)

    train_accuracies.append(epoch_train_accuracy)
    valid_accuracies.append(epoch_valid_accuracy)

    print(
        f"Epoch [{epoch+1}/{num_epochs}] "
        f"Train Loss: {epoch_train_loss:.4f} "
        f"Train Acc: {epoch_train_accuracy:.4f} "
        f"Valid Loss: {epoch_valid_loss:.4f} "
        f"Valid Acc: {epoch_valid_accuracy:.4f}"
    )

    # Early stopping
    if epoch_valid_loss < best_valid_loss:
```

```
            best_valid_loss = epoch_valid_loss
            patience_counter = 0

            torch.save(
                model.state_dict(),
                "/content/best_vgg16_model.pth"
            )

        else:

            patience_counter += 1

            if patience_counter >= patience:
                print("Early stopping triggered.")
                break

    print("Training completed!")
```

```
Epoch [1/3] Train Loss: 2.0582 Train Acc: 0.5375 Valid Loss: 0.8060 Valid Acc
Epoch [2/3] Train Loss: 0.8945 Train Acc: 0.7559 Valid Loss: 0.5979 Valid Acc
Epoch [3/3] Train Loss: 0.7473 Train Acc: 0.7869 Valid Loss: 0.5832 Valid Acc
Training completed!
```

```
model.load_state_dict(
    torch.load(
        "/content/best_vgg16_model.pth",
        map_location=device
    )
)

model = model.to(device)

print("Best model loaded successfully!")
```

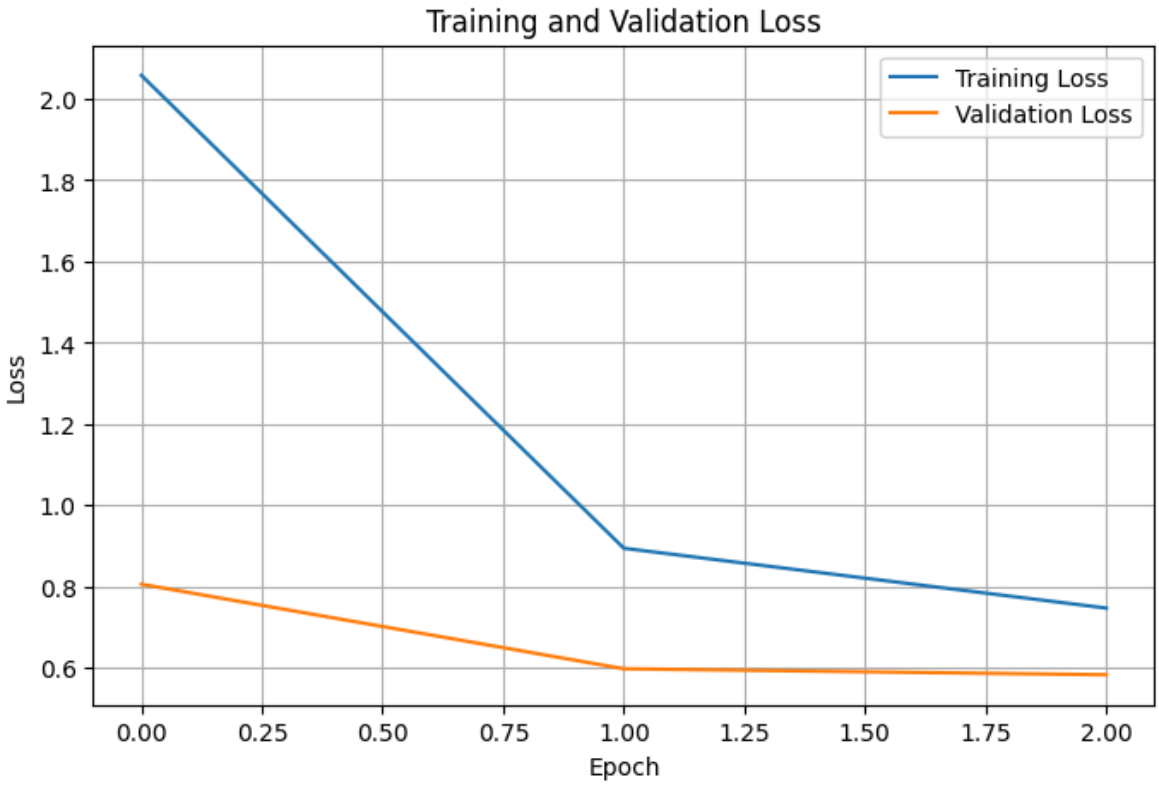
```
Best model loaded successfully!
```

```
plt.figure(figsize=(8, 5))

plt.plot(train_losses, label="Training Loss")
plt.plot(valid_losses, label="Validation Loss")

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training and Validation Loss")
plt.legend()
plt.grid()

plt.show()
```

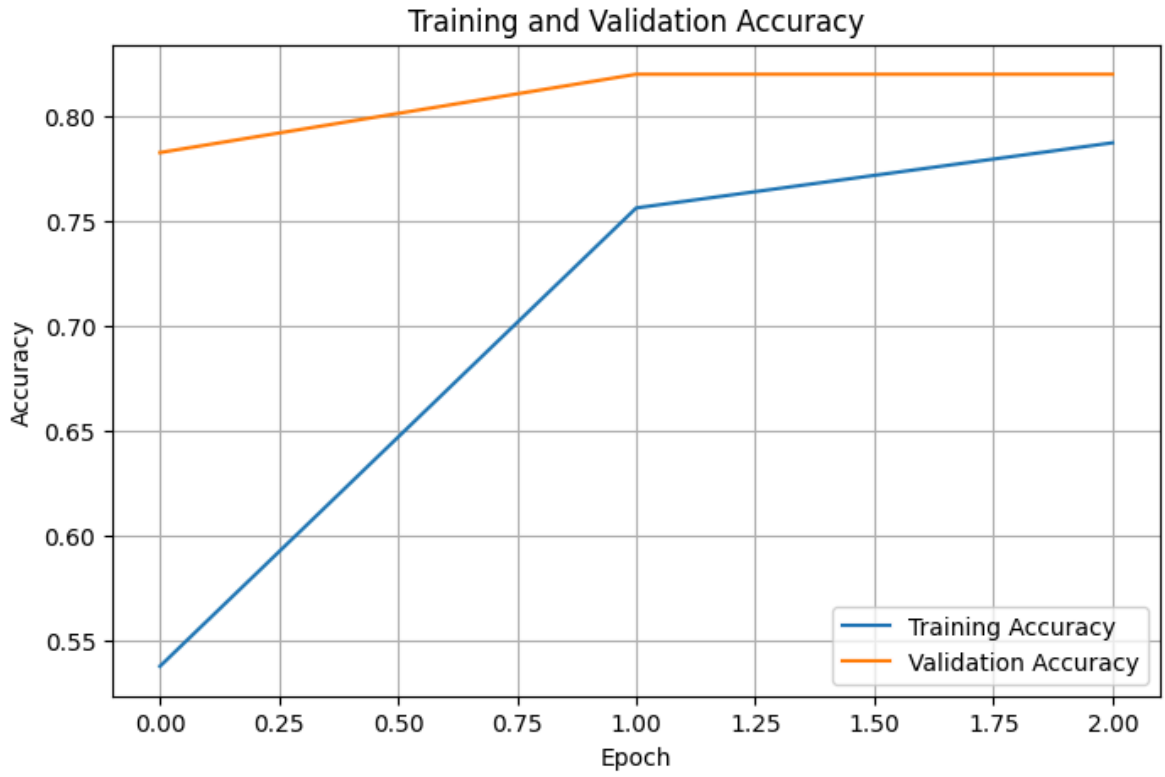



```
plt.figure(figsize=(8, 5))

plt.plot(train_accuracies, label="Training Accuracy")
plt.plot(valid_accuracies, label="Validation Accuracy")

plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Training and Validation Accuracy")
plt.legend()
plt.grid()

plt.show()
```




```
model.eval()

correct = 0
total = 0

with torch.no_grad():

    for images, targets in dataloaders["test"]:

        images = images.to(device)
        targets = targets.to(device)

        outputs = model(images)

        _, predicted = torch.max(outputs, 1)

        total += targets.size(0)

        correct += (predicted == targets).sum().item()

test_accuracy = correct / total

print("Test Accuracy:", test_accuracy)
print("Test Accuracy (%):", test_accuracy * 100)
```

```
Test Accuracy: 0.8174773999139044
Test Accuracy (%): 81.74773999139045
```

```
images, targets = next(iter(dataloaders["test"]))

images_gpu = images.to(device)

model.eval()

with torch.no_grad():
    outputs = model(images_gpu)
    _, predictions = torch.max(outputs, 1)

plt.figure(figsize=(14, 8))

for i in range(8):

    image = images[i].permute(1, 2, 0).numpy()

    mean = np.array([0.485, 0.456, 0.406])
    std = np.array([0.229, 0.224, 0.225])

    image = image * std + mean
    image = np.clip(image, 0, 1)

    plt.subplot(2, 4, i + 1)

    plt.imshow(image)

    actual = class_names[targets[i]]
    predicted = class_names[predictions[i]]

    plt.title(
        f"Actual: {actual}\nPredicted: {predicted}"
    )

    plt.axis("off")

plt.tight_layout()
plt.show()
```




```
print("=" * 50)
print("TRANSFER LEARNING - VGG16")
print("=" * 50)

print("Dataset       : Caltech-101")
```